Entrega 8 · verificación local (contenedores e infraestructura)

Equipo Windows con Docker Desktop y Smart App Control activo. Todo lo siguiente se **ejecutó** salvo lo indicado en "No ejecutado". Ningún recurso en la nube.

1. Imágenes y ambiente QA en contenedores

```
docker build -f docker/{api,web,worker,migrator}.Dockerfile -t medicore/<servicio>:local .   # 4
imágenes, sin errores
scripts/qa.sh init && scripts/qa.sh up && scripts/qa.sh seed
```

- Todos los servicios quedaron `healthy` (migrate terminó con código 0). La base **nueva** se migró desde cero con el paquete de EF (`LaboratorioYNotificaciones` fue la última migración aplicada).
- Defectos encontrados al ejecutarlo y corregidos (ver hallazgos H8-01 y H8-02): permisos del volumen de claves de la web y orden de los sembradores en una base nueva.

1b. Clon limpio

Se clonó la rama desde GitHub en una carpeta temporal y, usando solo lo versionado, se ejecutaron `scripts/qa.sh init`, `docker compose config` y la construcción de las 4 imágenes: todo correcto. (Esta comprobación detectó que `.env.qa.example` estaba ignorado por `.gitignore` y no se versionaba; se corrigió.)

2. Pruebas de humo de web y API (`scripts/qa.sh smoke`, a través del proxy)

11 comprobaciones, todas OK: liveness y readiness de la API (incluye PostgreSQL), la API exige autenticación (401), la web sirve `/Login` en español con CSP, la raíz redirige al inicio de sesión (302), la documentación OpenAPI no se expone en QA, inicio de sesión de Recepción por la API con los usuarios ficticios sembrados y listado de médicos autenticado.

Además, el spec de Cypress de Recepción (8 pruebas, Chrome) pasó **contra el ambiente QA en contenedores a través del proxy** (`MEDICORE_WEB_URL=http://localhost:8080`): 8/8. Cubre el inicio de sesión real de la web con la cookie cifrada cuyas claves viven en el volumen `web-keys`.

3. Persistencia (`scripts/qa-persistence.sh`)

Se crean por la API un paciente nuevo y un resultado de laboratorio (que dispara el evento por el outbox → worker → RabbitMQ → notificación del paciente) y se comprueban tres veces (inicial, tras `restart`, tras `down` + `up`): el paciente, el resultado y **una sola** notificación existen siempre. Se comprobó que los 6 volúmenes `medicore-qa_*` se conservan tras `down`. Resultado: **13/13 comprobaciones OK**. No se borró ningún volumen del desarrollo (`medicore_*`); el único `destroy` fue sobre los volúmenes `medicore-qa_*` que esta misma entrega había creado, para probar una instalación desde cero.

4. Puertos expuestos

`docker ps` del proyecto `medicore-qa`: solo el proxy publica puertos (127.0.0.1:8080 y 127.0.0.1:8443). PostgreSQL, Redis, RabbitMQ, API, web y worker no publican nada; la red `backend` es `internal` (sin salida a Internet ni al anfitrión).

5. App Android contra el ambiente local en contenedores

APK compilado con `scripts/android-build-linux.sh http://10.0.2.2:8080/` (el emulador ve el proxy del anfitrión), instalado en el emulador `medicore_pixel` (Android 35). Los 8 flujos Maestro se ejecutaron contra el ambiente QA (la preparación de datos usa la API del proxy y la base del Compose de QA):

- **Segunda ejecución completa: 8 de 8 aprobados** (`documentacion/evidencias/entrega-8-maestro-qa.md`).
- Primera ejecución: 6 de 8; los flujos 01 y 03 fallaron en 8-9 s por `DeviceServerDiedException` del driver de Maestro en el emulador (`device server died during deviceInfo`), no por la app ni por el ambiente (02 y 04-08 usaron el mismo inicio de sesión y pasaron). Se repitió la suite completa y pasó.
- Límite: un emulador; solo el tráfico por HTTP a `10.0.2.2` (la app no permite HTTP hacia otros hosts). Con Azure se necesitará HTTPS.

6. Pruebas xUnit después de los cambios de código

Cambió código (proyecto `MediCore.Worker`, reordenamiento de sembradores, cabeceras reenviadas y claves de protección de datos en API y web, modo `--seed-qa`). Se **volvió a ejecutar** `scripts/dotnet-linux.sh test MediCore.slnx`: **293 superadas** (44 unitarias, 50 de la lógica móvil, 199 de integración), 0 fallidas. No se agregaron pruebas xUnit nuevas para el worker ni para `--seed-qa`: se verificaron con el ambiente en contenedores (secciones 1 a 3).

7. Validación de archivos de Terraform, Ansible y Docker (`scripts/validate-iac.sh`, en contenedores oficiales)

Qué	Resultado
Terraform 1.12.2: `fmt -check`, `init -backend=false`, `validate` en `azure` y `bootstrap`	correcto
Ansible-core 2.21: `--syntax-check` de `site.yml`	correcto
`ansible-lint` (perfil *production*)	0 fallos (se corrigieron 2 al inicio)
`hadolint` en los 4 Dockerfile (umbral *warning*)	sin advertencias; solo avisos informativos DL3066 (UID numérico)
`docker compose config` del ambiente QA	válido

Esto valida archivos. NO es un despliegue en Azure.

8. No ejecutado

- `terraform plan` / `apply` / `destroy` (requieren suscripción y credenciales de Azure; no se solicitó ejecutarlos).
- `ansible-playbook` contra un servidor real (no hay servidor), HTTPS con certificado real (Let's Encrypt necesita un nombre DNS público), el clon del repositorio desde la VM y el disco de datos de Azure.

- Cypress de Médico y Laboratorio contra QA (sus tareas escriben en la base y la base de QA no se publica); Maestro/Cypress contra Azure.
- Escaneo de seguridad de las imágenes y del código (DevSecOps pendiente).